



RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES, BASAR

(Approved by AICTE, RGUKT - Basar) Nirmal District, Telangana – 504107

www.rgukt.ac.in

Bachelor of Technology

ELECTRONICS AND COMMUNICATION ENGINEERING

MINOR PROJECT REPORT

ADAPTIVE DUAL-MODE MULTIPLY- ACCUMULATE (MAC) UNIT FOR RISC-V CORE, POWER-PERFORMANCE-AREA TRADE-OFFS

Prepared by:

K SAI RAHUL – B210691 (30)

K MANOJ SRIVATSA – B210776 (34)

S ANAND BABU – B211313 (50)

Under the guidance of:

Ms. Sravanthi

Assistant Professor, RGUKT, Basar

Acknowledgement

We express our sincere gratitude to our project guide **Mrs. Sravanthi**, Assistant Professor, Department of ECE, RGUKT Basar, for her valuable guidance, continuous support, and constructive suggestions throughout the course of this project. We are grateful to **Mr. B. Upender Rao**, Head of the Department of ECE, RGUKT Basar, for providing the necessary infrastructure and facilities to successfully carry out this work. We also thank all the faculty members of the Department of ECE and the laboratory staff for their support during the development and testing phases. Finally, we sincerely thank our family and friends for their constant encouragement and moral support throughout this endeavour.

Table Of Contents

Acknowledgement	2
Abstract	4
Introduction	5
Methodology	6
Block Diagram	7
Results, Outputs and Graphs	11
Discussion	12
Advantages	13
Limitations	13
Future Scope	14
Conclusion	14
References	14
Appendix-A: Verilog Code	15
Appendix-B: Testbench	18

Abstract

This work presents an adaptive dual-mode Multiply–Accumulate (MAC) architecture that optimizes power, performance, and area (PPA) through intelligent runtime operation. The design integrates a high-speed single-cycle basic MAC and a low-power multi-cycle iterative MAC, implemented using 32-bit Verilog RTL and evaluated through an ASIC design flow.

The basic MAC achieves high performance at 100 MHz with a power consumption of 1.89 mW, while the iterative MAC reduces power consumption to 0.45 mW by utilizing a shift-and-add multiplication approach over multiple clock cycles. An adaptive control mechanism, based on input activity monitoring, dynamically selects the appropriate mode of operation. Additionally, a mode-locking mechanism ensures stable execution by preventing switching during active computation, thereby maintaining data integrity.

Under a representative workload, the adaptive system achieves an estimated average power of 1.03 mW, demonstrating significant energy savings compared to static architectures. The results validate that the proposed design effectively balances performance and energy efficiency, making it suitable for modern embedded systems, DSP applications, and power-constrained VLSI designs.

Introduction

The **Multiply-Accumulate (MAC) unit** is a fundamental building block in many digital systems, particularly in Digital Signal Processing (DSP), machine learning accelerators, and communication systems. Traditional MAC architectures are typically designed either for high performance or for low power, but not both simultaneously. This creates inefficiency when operating conditions change dynamically.

To address this limitation, this project presents an **Adaptive Dual-Mode MAC Architecture**, which intelligently switches between two modes of operation:

- A **Basic MAC**, which provides high-speed computation with a latency of one clock cycle.
- An **Iterative MAC**, which performs the same operation over multiple cycles with reduced hardware activity and lower power consumption.

Methodology

The proposed system is designed as an adaptive dual-mode MAC architecture that dynamically switches between performance and energy-efficient operation based on input activity. The implementation follows a structured ASIC design flow.

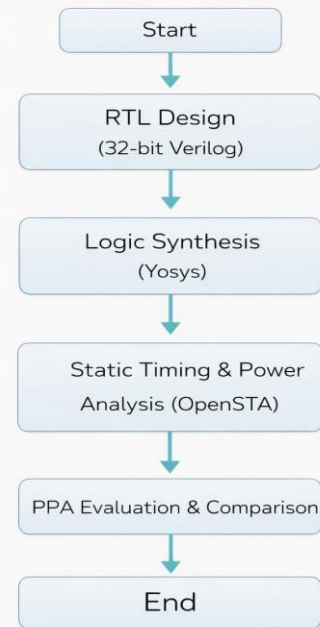
Initially, three RTL modules were developed in Verilog: a single-cycle basic MAC for high-performance operation, a multi-cycle iterative MAC for low-power computation, and an adaptive control unit responsible for mode selection. These modules were integrated to form a unified system capable of runtime adaptation.

The adaptive mechanism is based on input activity monitoring using a gap counter, which measures the interval between consecutive valid inputs. Based on this gap, the control unit selects the appropriate mode: the basic MAC for high-frequency inputs and the iterative MAC for low-frequency inputs. A mode-locking mechanism ensures that once a computation begins, the selected mode remains fixed until completion.

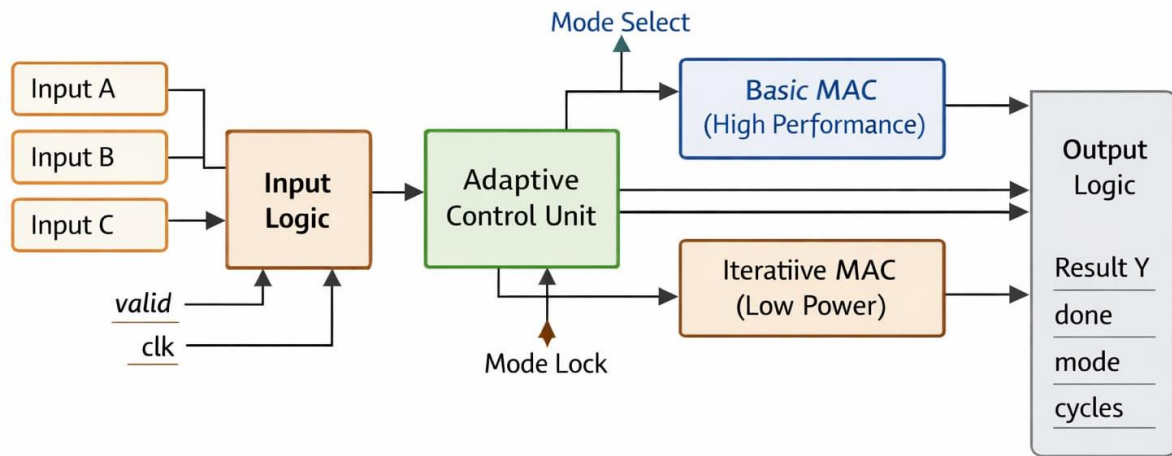
Functional verification was performed through simulation and waveform analysis to validate both computation correctness and adaptive behavior. The design was then synthesized using Yosys to generate gate-level netlists. Static timing and power analysis were carried out using OpenSTA under defined clock constraints and activity assumptions.

Finally, the basic, iterative, and adaptive architectures were evaluated and compared based on power, performance, and area (PPA) metrics to analyze trade-offs and demonstrate the effectiveness of workload-aware adaptive switching.

Methodology (ASIC Design Flow)



Block Diagram



The proposed system consists of four major blocks: input logic, adaptive control unit, MAC units, and output logic.

- The **input logic** receives operands (A, B, C) along with the valid signal and forwards them for processing.
- The **adaptive control unit** monitors the input arrival pattern using a gap counter and generates the mode selection signal. It also ensures mode locking during computation.
- Based on the selected mode, either the **basic MAC (high performance)** or the **iterative MAC (low power)** is activated.
- The **output logic** collects the result from the selected MAC unit and generates output signals such as Y, done, mode, and cycles.

This modular structure enables efficient switching between performance and energy-efficient computation.

Working Explanation

*Refer to [Appendix-A: Verilog Code](#) & [Appendix-B: Testbench](#)

The operation of the adaptive dual-mode MAC system is explained in the following steps:

1. Fundamental Operation

The system performs the standard MAC computation:

$$Y = (A \times B) + C$$

To improve efficiency, the system incorporates two computation modes:

- Basic MAC (Performance Mode): Executes the operation in a single clock cycle
- Iterative MAC (Energy Mode): Executes the operation over multiple clock cycles using a shift-and-add approach

The system automatically selects the appropriate mode depending on input conditions.

2. Input Reception and Latching

The system receives three operands (32-bit A, B, C) along with a valid control signal, which indicates the arrival of new data.

When valid = 1:

- The inputs are captured and stored internally
- The system enters a processing state
- A new MAC operation is initiated

3. Input Activity Detection (Gap Counter Mechanism)

To determine how frequently inputs are arriving, the system uses a gap counter mechanism:

- The counter increments during periods when no input is received
- It resets whenever new input arrives

This allows the system to measure the time gap between consecutive inputs.

Interpretation:

- Small gap $\rightarrow$ High input frequency
- Large gap $\rightarrow$ Low input frequency

4. Adaptive Mode Selection

The system compares the measured gap with a predefined threshold to decide the mode of operation:

- Small gap (frequent inputs) $\rightarrow$ Basic MAC (Performance Mode)
- Large gap (infrequent inputs) $\rightarrow$ Iterative MAC (Energy Mode)

This decision is stored internally and determines how the computation will proceed.

5. Mode Locking Mechanism

Once a mode is selected:

- It remains fixed throughout the computation
- No switching is allowed during execution

This ensures:

- Stable operation
- Prevention of computation errors
- Data integrity

The system only re-evaluates the mode after the current operation is completed.

6. Operation of Basic MAC (Performance Mode)

In this mode:

- The MAC operation is performed directly in hardware
- The result is computed in a single clock cycle
- The output is generated immediately

This mode is ideal for scenarios where inputs arrive rapidly and high-speed processing is required.

7. Operation of Iterative MAC (Energy Mode)

In this mode:

- Multiplication is performed using a shift-and-add algorithm
- The computation is carried out over multiple clock cycles (typically 32 cycles for 32-bit inputs)
- After multiplication, the addition with operand C is performed

This approach:

- Reduces switching activity
- Minimizes power consumption
- Is suitable for low input data rates

8. Output Generation

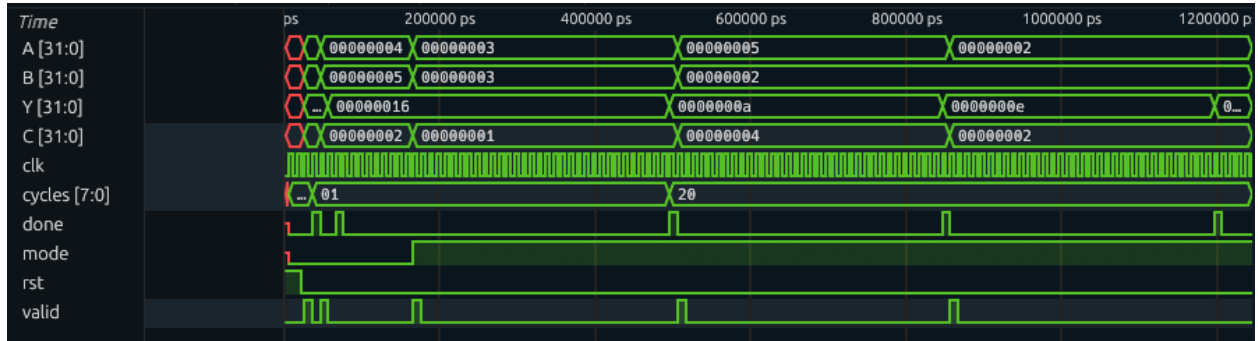
After the computation is completed, the system produces:

- Y → Final result of the MAC operation
- done → Indicates completion of computation
- mode → Specifies which mode was used
- cycles → Indicates the number of clock cycles taken

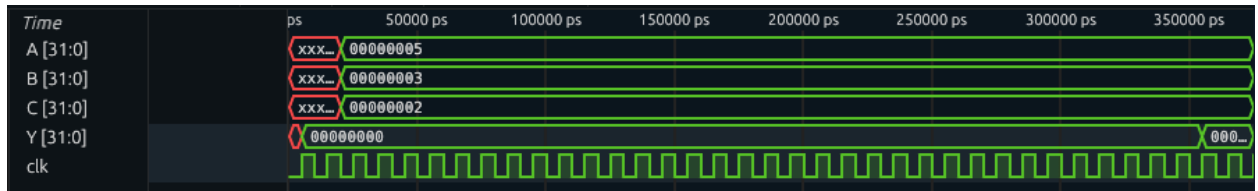
9. Adaptive System Behavior

Input Activity	Mode Selected	Behavior
High	Basic MAC	Fast computation (1 cycle)
Low	Iterative MAC	Energy-efficient computation

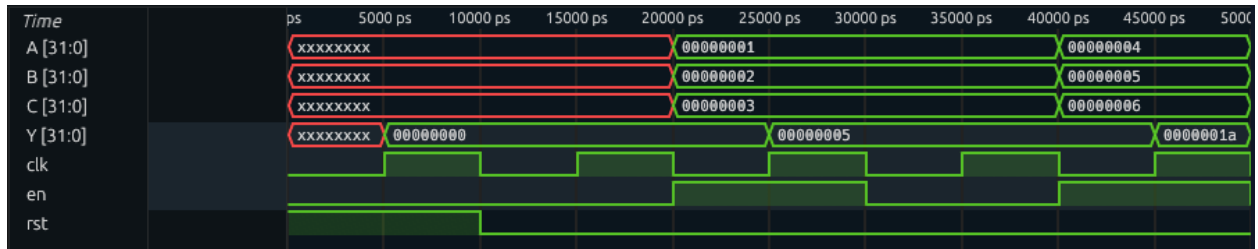
Results, Outputs and Graphs



Dual Mode MAC



Iterative MAC



Basic MAC

```
OpenLane Container (ff5509f):openlane$ sta
OpenSTA 2.4.0 4la5leaf4c Copyright (c) 2021, Parallax Software, Inc.
License GPLv3: GNU GPL version 3 <http://gnu.org/licenses/gpl.html>

This is free software, and you are free to change and redistribute it
under certain conditions; type 'show_copyright' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type 'show_warranty'.
% cd ~/home/manoj/projects/mac_project
% read_liberty lib/sky130.lib
1
% read_verilog mac_basic_sta.v
1
% link_design mac_basic
1
% create_clock -period 10 [all_inputs]
% report_worst_slack
worst slack 0.16
% report_power
Group          Internal Power  Switching Power  Leakage Power  Total Power (Watts)
-----
Sequential      6.12e-04    6.98e-05    1.08e-09    6.82e-04    36.1%
Combinational    7.50e-04    4.54e-04    7.77e-09    1.20e-03    63.9%
Macro           0.00e+00    0.00e+00    0.00e+00    0.00e+00    0.0%
Pad             0.00e+00    0.00e+00    0.00e+00    0.00e+00    0.0%
Total           1.36e-03    5.24e-04    8.85e-09    1.89e-03    100.0%
%
```

Basic MAC Power Report

```
Warning: set_input_delay relative to a clock defined on the same port/pin not allowed.
Timing Report (ITERATIVE MAC)
Worst slack 2.64
Error: report_checks -max_paths is not a known keyword or flag.
Power Report
Group          Internal Power  Switching Power  Leakage Power  Total Power (Watts)
-----
Sequential      3.80e-04    1.05e-05    1.15e-09    3.91e-04    86.1%
Combinational    3.40e-05    2.91e-05    1.50e-09    6.31e-05    15.9%
Macro           0.00e+00    0.00e+00    0.00e+00    0.00e+00    0.0%
Pad             0.00e+00    0.00e+00    0.00e+00    0.00e+00    0.0%
Total           4.10e-04    3.96e-05    2.72e-09    4.50e-04    100.0%
END OF REPORT
```

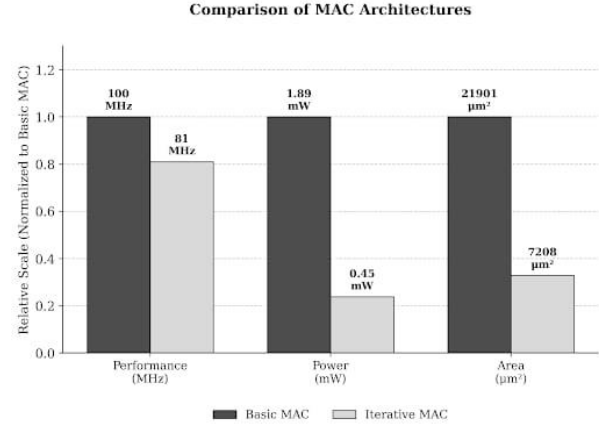
Iterative MAC Power Report

```

VCD info: dumpfile wave.vcd opened for output.
MODE=  BASIC | A=2 B=3 C=1 | Y=7 | CYCLES=1
MODE=  BASIC | A=4 B=5 C=2 | Y=22 | CYCLES=1
MODE=ITERATIVE | A=3 B=3 C=1 | Y=10 | CYCLES=32
MODE=ITERATIVE | A=5 B=2 C=4 | Y=14 | CYCLES=32
MODE=ITERATIVE | A=2 B=2 C=2 | Y=6 | CYCLES=32

```

Terminal Output



Mode	Power	Performance	Area
Basic	1.89mW	100MHz	21k μm^2
Iterative	0.45mW	81MHz	7.2k μm^2

Discussion

From the obtained results:

- The **basic MAC** produces output in a single clock cycle, demonstrating high-speed performance.
- The **iterative MAC** takes 32 clock cycles, confirming its multi-cycle energy-efficient operation.
- The mode switching occurs only after the completion of the current operation, ensuring stable and error-free execution.

The power analysis indicates that:

- The basic MAC consumes higher power due to parallel computation.
- The iterative MAC significantly reduces power consumption by spreading computation over multiple cycles.

Thus, the proposed design effectively achieves a **power-performance trade-off**, making it suitable for applications with varying workload conditions.

Advantages

- **Adaptive Operation:** Automatically switches between performance and energy modes based on input frequency.
- **High-Speed Processing:** Basic MAC provides results in a single clock cycle, suitable for real-time applications.
- **Energy Efficiency:** Iterative MAC reduces power consumption by spreading computation over multiple cycles.
- **Deterministic Latency:** Ensures fixed execution times (1 cycle and 32 cycles), enabling predictable system behavior.
- **Improved PPA Trade-off:** Effectively balances power, performance, and area compared to conventional static designs.

Limitations

- **Increased Design Complexity:** Additional control logic and dual MAC units make the system more complex.
- **Higher Hardware Resource Usage:** Requires more area due to integration of two MAC architectures.
- **Latency in Iterative Mode:** Multi-cycle operation (32 cycles) may not be suitable for time-critical applications.
- **Threshold Sensitivity:** Performance depends on proper selection of the gap threshold value.
- **Limited Power Optimization Techniques:** Advanced techniques like clock gating are not implemented in this design.

Future Scope

- **Dynamic Threshold Adjustment:** Implement adaptive threshold selection based on runtime workload conditions.
- **Advanced Power Reduction Techniques:** Integrate clock gating, power gating, or voltage scaling.
- **Pipeline-Based MAC Extension:** Add a pipelined MAC mode to improve throughput and flexibility.
- **Hardware Implementation:** Deploy the design on FPGA/ASIC for real-time validation and optimization.
- **Scalability Enhancement:** Extend the architecture to support higher bit-width operations (e.g., 64-bit MAC).

Conclusion

The project demonstrates an **adaptive MAC system** that balances performance and energy efficiency. It ensures **safe and accurate operation** using controlled mode switching. The design is suitable for **modern embedded, Digital Signal Processing and VLSI applications** with varying data rates.

References

- Verilog HDL documentation and FPGA/ASIC design resources.
- IEEE Paper: [Multiply-Accumulate Unit by Integrating Additions and Accumulations Into Partial Product Reduction Process](#)
- ChatGPT, YouTube.

Appendix-A: Verilog Code

```
`timescale 1ns/1ps
module mac_dual (
    input clk,
    input rst,
    input valid,
    input [31:0] A, B, C,
    output reg [31:0] Y,
    output reg done,
    output reg mode, //0-Basic, 1-ITERATIVE
    output reg [7:0] cycles
);
    // -----
    // INPUT LATCH
    // -----
    reg [31:0] A_reg, B_reg, C_reg;
    // -----
    // GAP DETECTION
    // -----
    reg [7:0] gap_counter;
    reg mode_req;
    parameter THRESHOLD = 5;
    always @(posedge clk) begin
        if (rst)
            gap_counter <= 0;
        else if (valid)
            gap_counter <= 0;
        else
            gap_counter <= gap_counter + 1;
    end
    always @(posedge clk) begin
        mode_req <= (gap_counter < THRESHOLD) ? 0 : 1;
    end
    // -----
    // CONTROL
    // -----
    reg busy;
    always @(posedge clk) begin
        if (rst) begin
            busy <= 0;
            mode <= 0;
        end
        else if (valid && !busy) begin
            busy <= 1;
            mode <= mode_req;
            A_reg <= A;
            B_reg <= B;
            C_reg <= C;
        end
        else if (done)
            busy <= 0;
    end
end
```

```

// -----
// BASIC MAC (EXACT 1 CYCLE)
// -----
reg basic_exec;
always @(posedge clk) begin
    if (rst) begin
        basic_exec <= 0;
    end
    else begin
        basic_exec <= 0;
        if (valid && !busy && mode_req==0) begin
            Y <= (A * B) + C;
            basic_exec <= 1;
        end
    end
end

// -----
// ITERATIVE MAC (EXACT 32)
// -----
reg [63:0] product;
reg [31:0] multiplicand, multiplier;
reg [5:0] count;
reg iter_active;
always @(posedge clk) begin
    if (rst) begin
        product <= 0;
        multiplicand <= 0;
        multiplier <= 0;
        count <= 0;
        iter_active <= 0;
    end
    else begin
        // START
        if (valid && !busy && mode_req==1) begin
            product <= 0;
            multiplicand <= A;
            multiplier <= B;
            count <= 0;
            iter_active <= 1;
        end
        // RUN (exact 32)
        else if (iter_active && count < 32) begin
            if (multiplier[0])
                product <= product + multiplicand;
            multiplicand <= multiplicand << 1;
            multiplier <= multiplier >> 1;
            count <= count + 1;
        end
        // DONE
        else if (iter_active && count == 32) begin
            Y <= product[31:0] + C_reg;
            iter_active <= 0;
        end
    end
end

```



```

        end
    end
    // -----
    // DONE + EXACT CYCLE COUNT
    // -----
    always @(posedge clk) begin
        if (rst) begin
            done <= 0;
            cycles <= 0;
        end
        else begin
            done <= 0;
            // BASIC → exactly 1 cycle
            if (basic_exec) begin
                done <= 1;
                cycles <= 1;
            end
            // ITERATIVE → exactly 32 cycles
            else if (mode==1 && count == 32 && iter_active) begin
                done <= 1;
                cycles <= 32;
            end
        end
    end
end
endmodule

```

Appendix-B: Testbench

```
`timescale 1ns/1ps
module mac_dual_tb;
    reg clk, rst, valid;
    reg [31:0] A, B, C;
    wire [31:0] Y;
    wire done;
    wire mode;
    wire [7:0] cycles;
    mac_dual uut (
        .clk(clk),
        .rst(rst),
        .valid(valid),
        .A(A), .B(B), .C(C),
        .Y(Y),
        .done(done),
        .mode(mode),
        .cycles(cycles)
    );
    // CLOCK
    always #5 clk = ~clk;
    // TASK
    task send_input(input [31:0] a, b, c);
    begin
        @(posedge clk);
        valid = 1;
        A = a; B = b; C = c;
        @(posedge clk);
        valid = 0;
        wait(done);
        $display("MODE=%s | A=%0d B=%0d C=%0d | Y=%0d | CYCLES=%0d",
            mode ? "ITERATIVE" : "BASIC",
            a, b, c, Y, cycles);
    end
    endtask

    initial begin
        clk = 0; rst = 1; valid = 0;
        #20 rst = 0;
        // FAST → BASIC
        send_input(2,3,1);
        send_input(4,5,2);
        // DELAY → ITERATIVE
        #100;
        send_input(3,3,1);
        send_input(5,2,4);
        // FAST → BASIC again
        send_input(2,2,2);
        #50;
        $finish;
    end
endmodule
```